Generic Programming

Review : Wrapper Classes

Wrapper classes provide a way to use primitive data types (int, boolean, etc..) as objects.

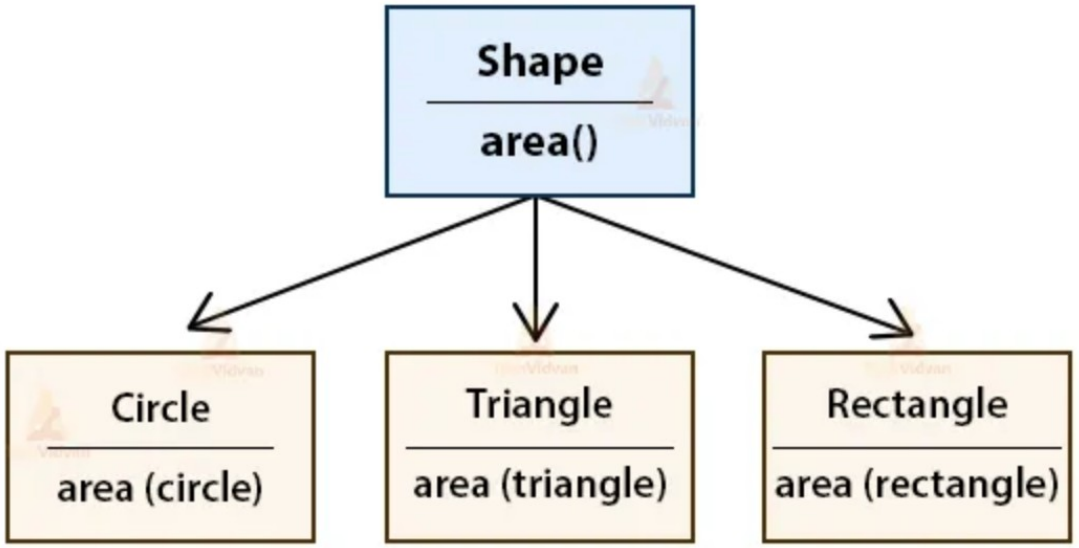
Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Sometimes you must use wrapper classes, for example when working with Collection objects, such as `ArrayList`, where primitive types cannot be used (the list can only store objects):

```
ArrayList<int> myNumbers = new ArrayList<int>(); // Invalid
ArrayList<Integer> myNumbers = new ArrayList<Integer>(); // Valid
```

Review : Polymorphism

The ability for the same code to be used with several different types of objects and for the **code to behave differently** depending on the actual type of object used.



JAVA knows which area method to call based on the type of the object. This is usually resolved during runtime.

Method overloading – resolved during compile time (Compile time polymorphism)
Method overriding – resolved during runtime (run time polymorphism)

Generic Programming

```java
// Find the minimum of three **ints**
public static Integer tripleMinInt(Integer item1, Integer item2, Integer item3) {
   Integer minVal;

   minVal = item1;

   if (item2.compareTo(minVal) < 0) {
      minVal = item2;
   }
   if (item3.compareTo(minVal) < 0) {
      minVal = item3;
   }
   return minVal;
}
```

```java
// Find the minimum of three **chars**
public static Character tripleMinChar(Character item1, Character item2, Character item3) {
   Character minVal;

   minVal = item1;

   if (item2.compareTo(minVal) < 0) {
      minVal = item2;
   }
   if (item3.compareTo(minVal) < 0) {
      minVal = item3;
   }
   return minVal;
}
```

Generic Programming means writing code that can be reused for objects of many different types.

Generic Methods

generic method to exchange the positions of two different elements in an array.

```
public final class Algorithm {
    public static <T> void swap(T[] a, int i, int j) {
        T temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}
```

Generic Classes

Java Generic generic classes enable programmers to specify, with a single method declaration, a set of related methods, or with a single class declaration, a set of related types, respectively.

Generics also provide compile-time type safety that allows programmers to catch invalid types at compile time.

A Generic class can have multiple type parameters.

Type Parameter Naming Conventions

By convention, type parameter names are named as single, uppercase letters so that a type parameter can be distinguished easily with an ordinary class or interface name. Following is the list of commonly used type parameter names −

- **E** − Element, and is mainly used by Java Collections framework.
- **K** − Key, and is mainly used to represent parameter type of key of a map.
- **V** − Value, and is mainly used to represent parameter type of value of a map.
- **N** − Number, and is mainly used to represent numbers.
- **T** − Type, and is mainly used to represent first generic type parameter.
- **S** − Type, and is mainly used to represent second generic type parameter.
- **U** − Type, and is mainly used to represent third generic type parameter.
- **V** − Type, and is mainly used to represent fourth generic type parameter.

Type Safety

Generics make errors to appear compile time than at run time (It's always better to know problems in your code at compile time rather than making your code fail at run time).

Suppose you want to create an ArrayList that store name of students and if by mistake programmer adds an integer object instead of a string, the compiler allows it. But, when we retrieve this data from ArrayList, it causes problems at runtime.

```
import java.util.*;

class Test
{
    public static void main(String[] args)
    {
        // Creatinga an ArrayList without any type specified
        ArrayList al = new ArrayList();

        al.add("Sachin");
        al.add("Rahul");
        al.add(10); // Compiler allows this

        String s1 = (String)al.get(0);
        String s2 = (String)al.get(1);

        // Causes Runtime Exception
        String s3 = (String)al.get(2);
    }
}
```

Sort Objects

Java provides two interfaces to sort objects using data members of the class:

1.Comparable
2.Comparator

A comparable object is capable of comparing itself with another object. The class itself must implements the **java.lang.Comparable** interface to compare its instances.

Unlike Comparable, Comparator is external to the element type we are comparing. It’s a separate class. We create multiple separate classes (that implement Comparator) to compare by different members.

Collections class has a second sort() method and it takes Comparator. The sort() method invokes the compare() to sort objects.

In-class activity

Define a generic method called **checkOrder()** that checks if four items are in ascending, neither, or descending order.

The method should return -1 if the items are in ascending order,
0 if the items are unordered, and
1 if the items are in descending order.

The program reads four items and outputs if the items are ordered. The items can be different types, including integers, Strings, characters, or doubles.

In-class activity (Solution)

```
public class Main {
   // TODO: Define a generic method called checkOrder() that
   //       takes in four variables of generic type as arguments.
   //       The return type of the method is integer
   public static <T extends Comparable<T>>  int checkOrder(T item1, T item2, T item3, T item4) {

      // Check the order of the input: return -1 for ascending,
      // 0 for neither, 1 for descending
      if ((item1.compareTo(item2) < 0)
            && (item2.compareTo(item3) < 0)
            && (item3.compareTo(item4) < 0)) {
         return -1;
      }
      else if ((item1.compareTo(item2) > 0)
            && (item2.compareTo(item3) > 0)
            && (item3.compareTo(item4) > 0)) {
         return 1;
      }
      else {
         return 0;
      }
   }

   public static void main(String[] args) {
      // Check order of four strings
      System.out.println("Order: " + checkOrder(1,2,3,4));
      // Check order of four doubles
      System.out.println("Order: " + checkOrder(8.3, 7.5, 6.5, 6.1));
      //check order of String
      System.out.println("Order: " + checkOrder("cat","dog","fish","goat"));
   }
}
```